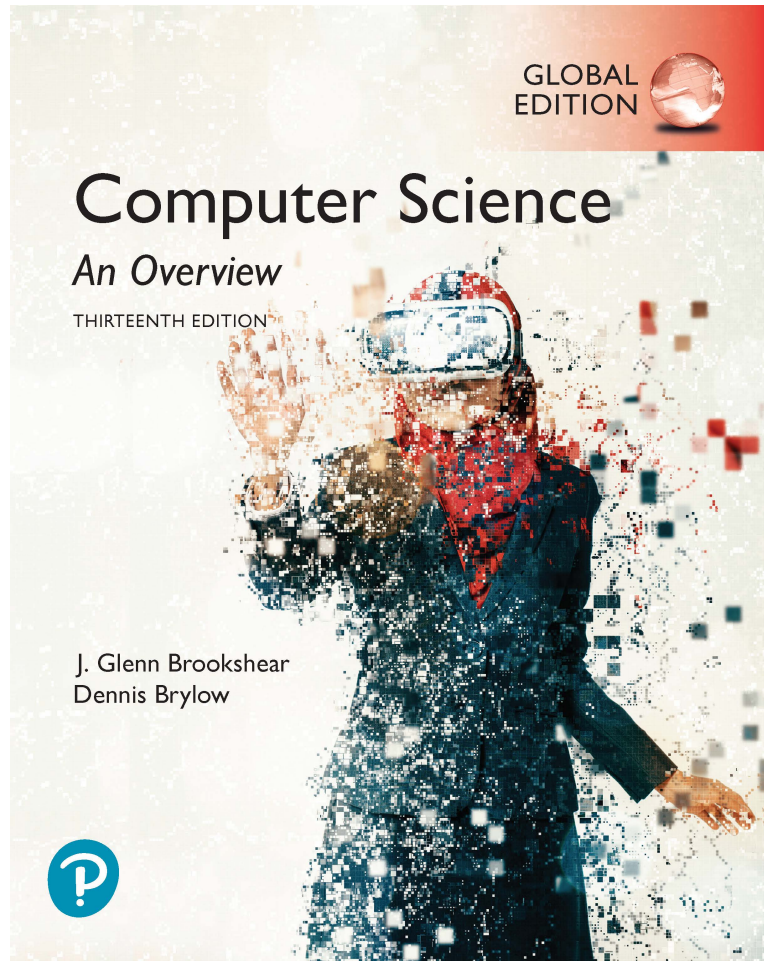


Computer Science An Overview

13th Edition, Global Edition



Chapter 8

Data Abstractions

資料抽象化

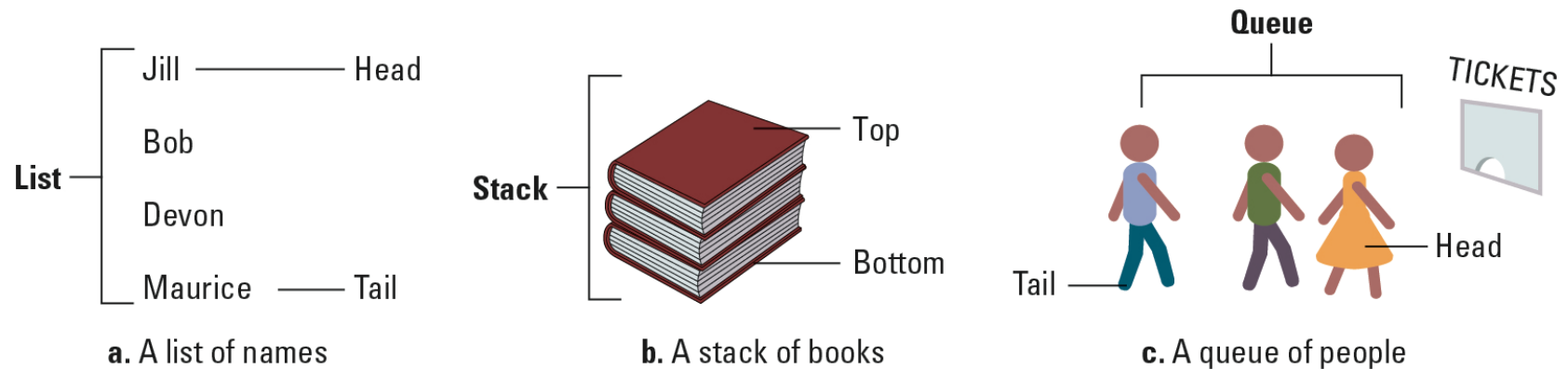
Chapter 8: Data Abstractions

- 8.1 Basic Data Structures
- 8.2 Related Concepts
- 8.3 Implementing Data Structures
- 8.4 A Short Case Study
- 8.5 Customized Data Types
- 8.6 Classes and Objects
- 8.7 Pointers in Machine Language

8.1 Basic Data Structures

- Arrays
- Aggregates 聚合結構
- Lists
 - Stacks
 - Queues
- Trees

Figure 8.1 Lists, stacks, and queues



Terminology for Arrays 有著相同資料形態的資料塊

- **Array:** A block of data whose entries are of same type
二維數組
- A two **dimensional** array consists for rows and columns
- **Indices** are used to identify positions
(Index) 識別位置

Terminology for Aggregates

- **Aggregate:** A block of data items that might be of different type or sizes → 可以有不同資料形態
- Each data item is called a **field**
- Fields are usually accessed by name

Terminology for Lists

- **List:** A collection of data whose entries are arranged sequentially — 按照顺序排列的资料集
- **Head:** The beginning of the list
- **Tail:** The end of the list

Terminology for Stacks

P. 439

- **Stack:** A list in which entries are removed and inserted only at the head
- **LIFO:** Last-in-first-out 後進先出
- **Top:** The head of list (stack)
- **Bottom** or **base:** The tail of list (stack)
- **Pop:** To remove the entry at the top
- **Push:** To insert an entry at the top

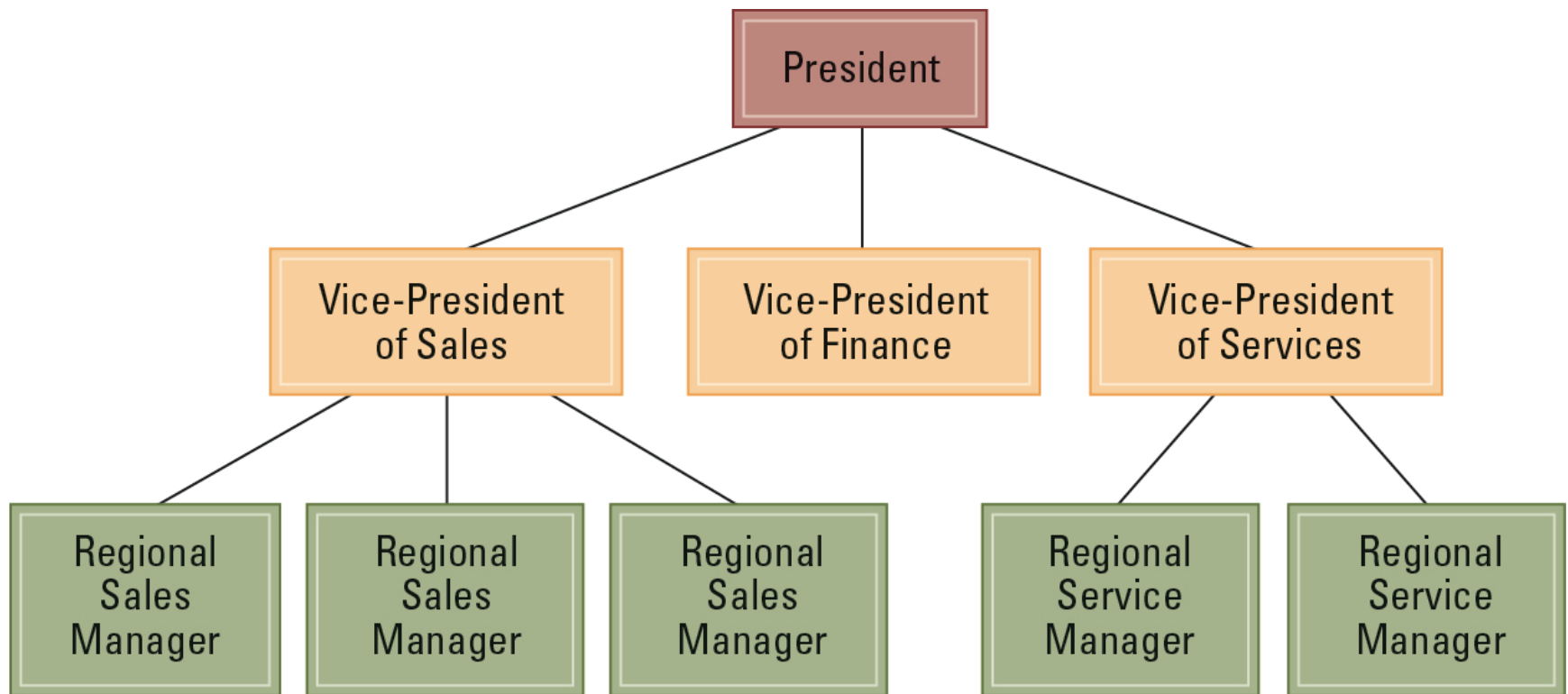
Stacks: 所有的 Remove & insert 都只會在 Head (Top) 進行!

Terminology for Queues

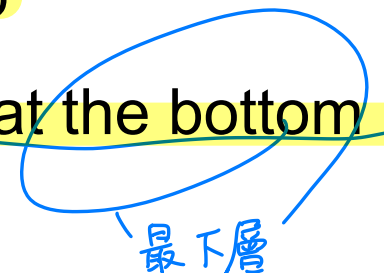
- **Queue:** A list in which entries are removed at the head and are inserted at the tail
- **FIFO:** First-in-first-out

先進先出

Figure 8.2 An example of an organization chart



Terminology for a Tree

- **Tree:** A collection of data whose entries have a hierarchical organization — 具有層次結構的資料集合
 - 節點 **Node:** An entry in a tree
 - 根節點 **Root node:** The node at the top
 - **Terminal or leaf node:** A node at the bottom
(端節點) 葉節點
- 

Terminology for a Tree (continued)

父節点(父親)

- **Parent:** The node immediately above a specified node

子節点(小孩)

- **Child:** A node immediately below a specified node

祖先

- **Ancestor:** Parent, parent of parent, etc.

後代

- **Descendent:** Child, child of child, etc.

兄弟

- **Siblings:** Nodes sharing a common parent

兩節点有一共同 Parent

Terminology for a Tree (continued)

二元樹

- **Binary tree:** A tree in which every node has at most two children

每個節點最多有兩個子節點

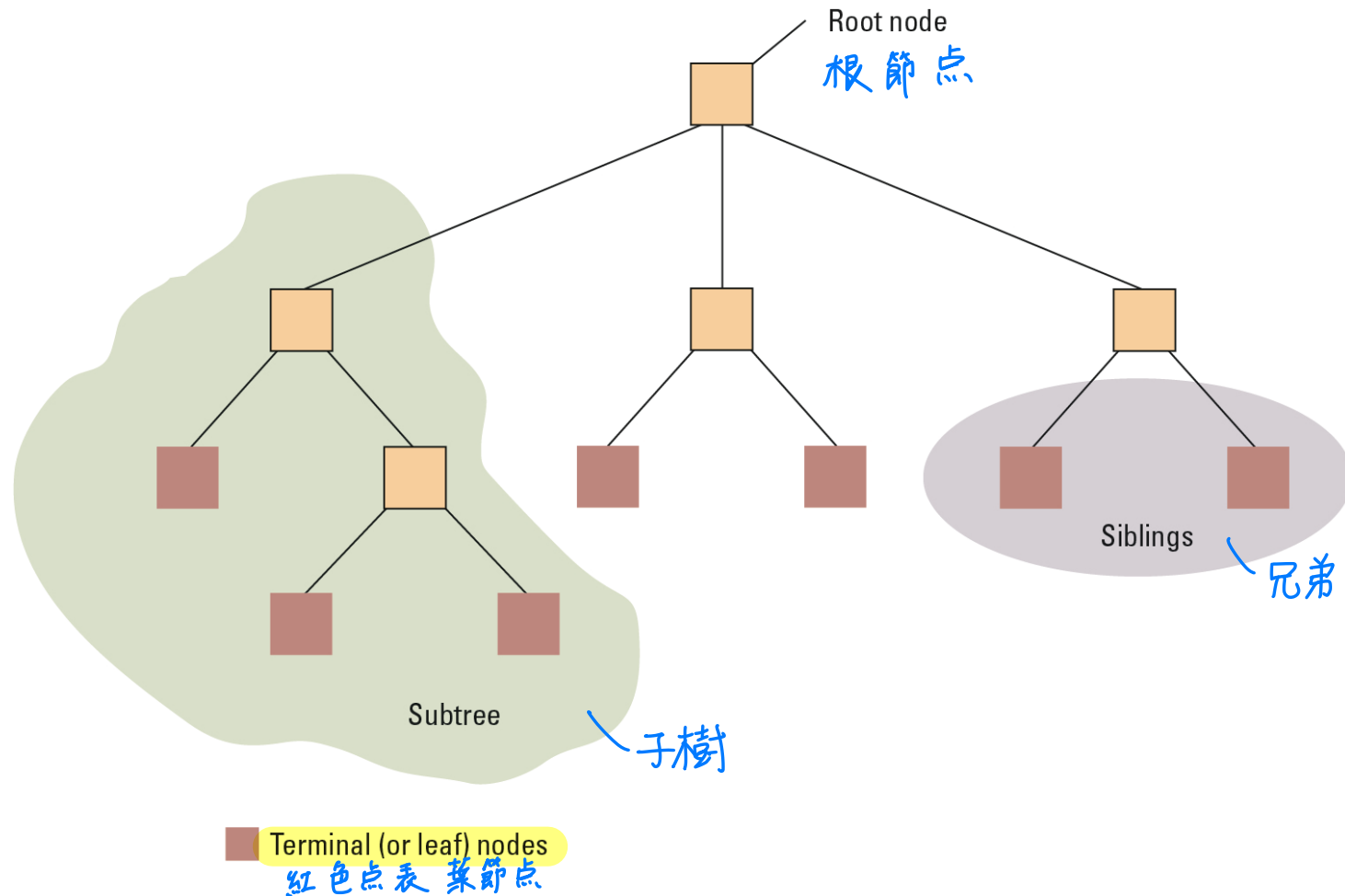
深度

- **Depth:** The number of nodes in longest path from root to leaf

從根→葉最長路徑

Link : 課 P.442

Figure 8.3 Tree terminology



8.2 Related Concepts

- Abstraction 抽象化

user 不受實際儲存的用戶影響

- Shield *users* (application software) from details of actual data storage

靜態 動態 結構

- Static vs. Dynamic Structure

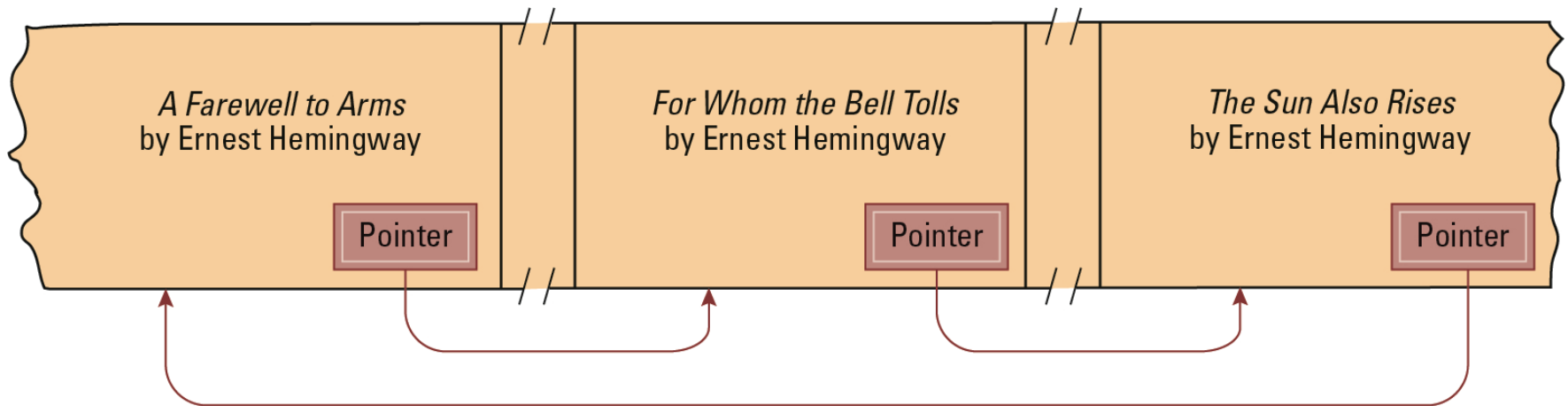
- Does the shape and size change over time?

資料結構的形態、大小 是否隨著時間改變

- **Pointer** 指標

- A storage area that contains the address where data is stored 記憶體位置

Figure 8.4 Novels arranged by title but linked according to authorship



8.3 Implementing Data Structures

- High Level Languages provide certain data structures as primitives 部分高階語言已經提供一些資料結構
- These data structures are translated into machine-language instructions to manipulate data stored in main memory

Storing Arrays

儲存(實作) 數組

特定單元の記憶体位置可以被計算!

- Memory address of a particular cell can be computed

- Row-major order** versus **column major order**

- An Address polynomial describes how to access the data in a certain array element

Address Polynomial 描述如何存取特定元素的資料

<key> malloc() の利用法

Address Polynomial: 課 P.448

$$x + (c \times (i - 1)) + (j - 1)$$

Address Polynomial 等同於以 Row-major 存儲的 2D Array.

The preceding calculation can be generalized to obtain a formula for converting references in terms of row and column positions into actual memory addresses. In particular, if we let c represent the number of columns in an array (which is the number of entries in each row), then the address of the entry in the i th row and j th column will be

$$x + (c \times (i - 1)) + (j - 1)$$

where x is the address of the cell containing the entry in the first row and first column. That is, we must move beyond $i - 1$ rows, each of which contains c entries, to reach the i th row and then $j - 1$ more entries to reach the j th entry in this row. In our prior example, $c = 5$, $i = 3$, and $j = 4$, so if the array were stored starting at address x , then the entry in the third row, fourth column

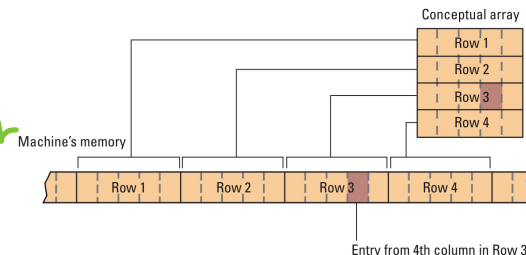


Figure 8.5 The array of temperature readings stored in memory starting at address x

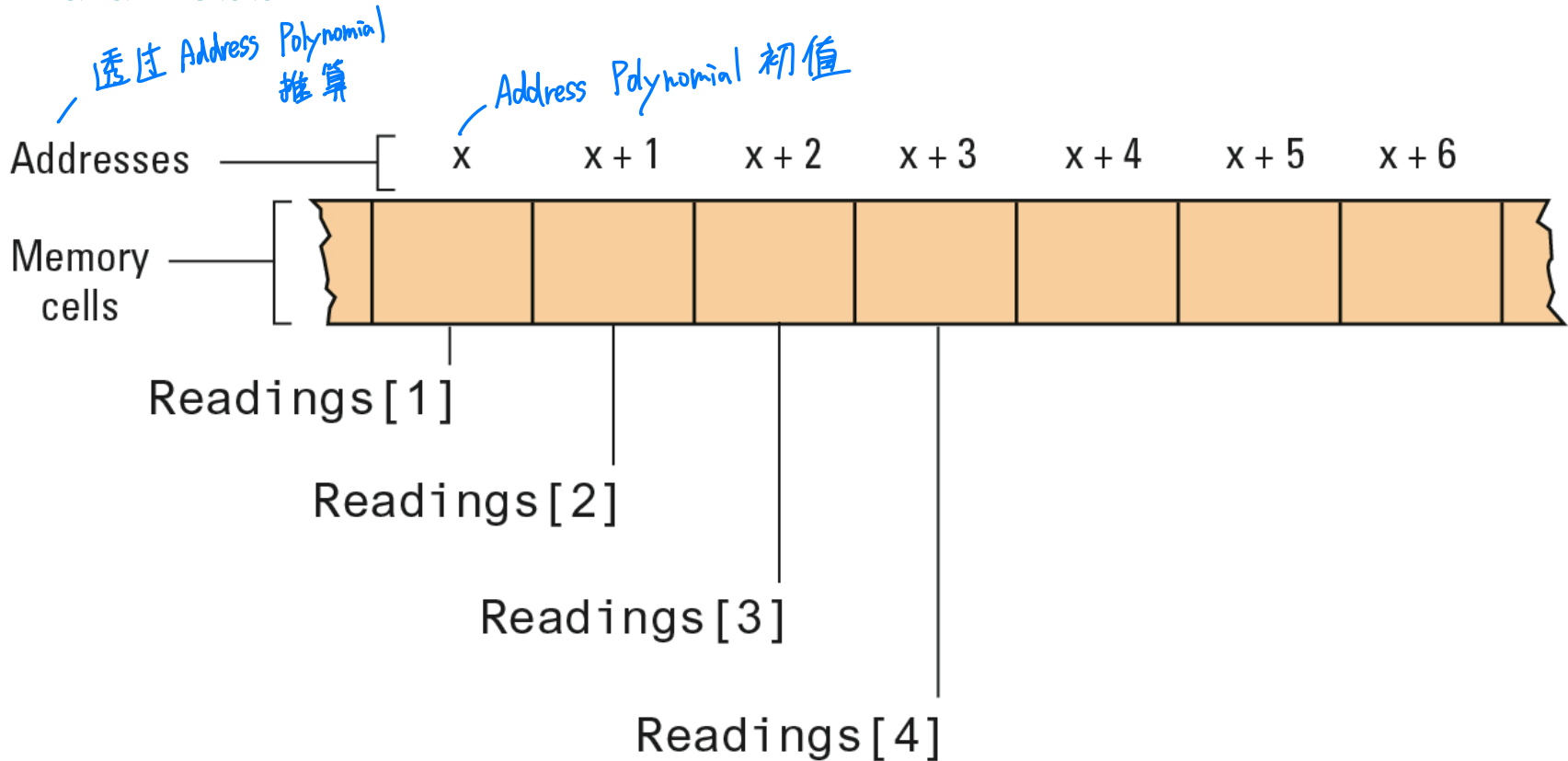
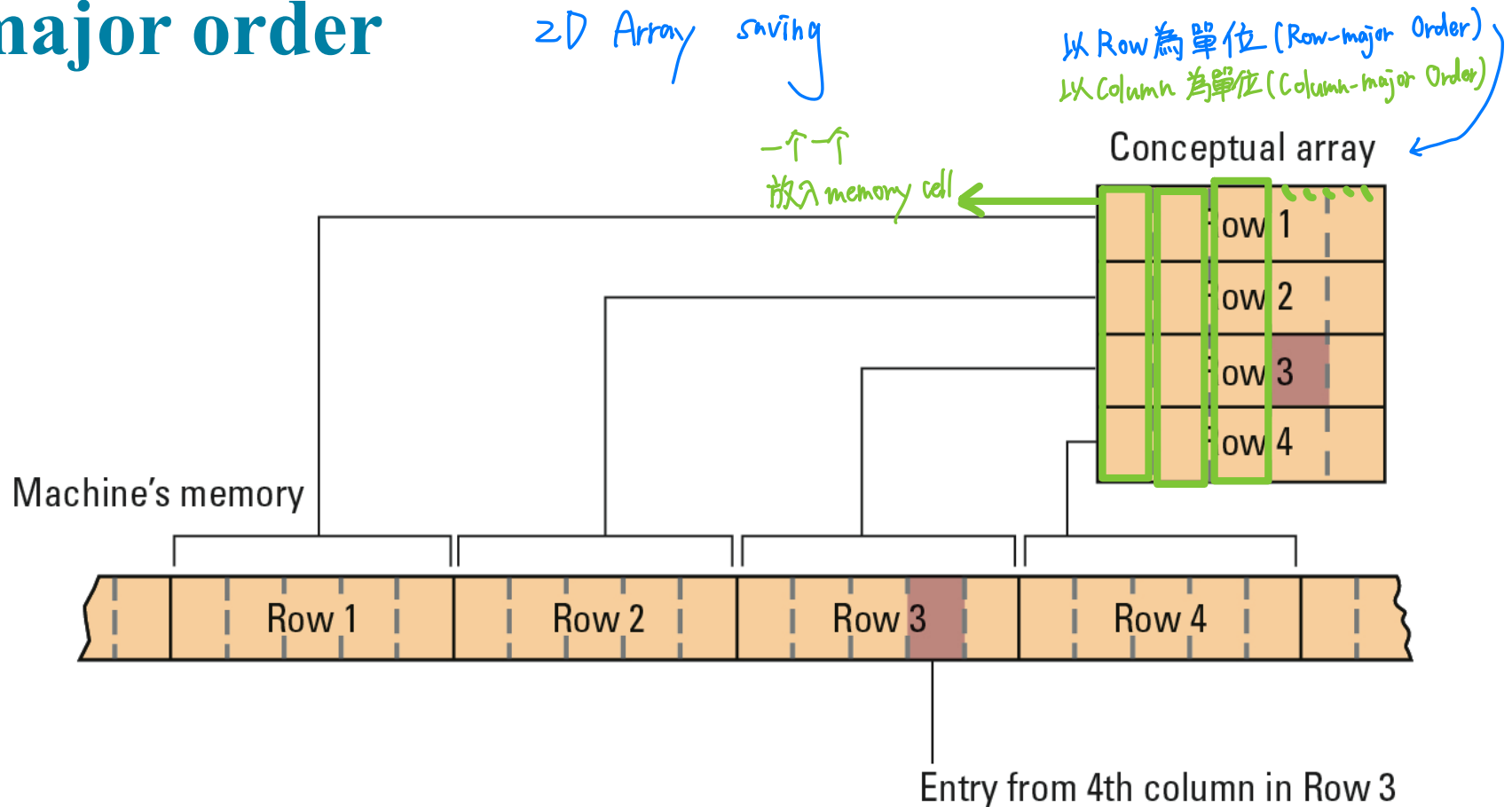


Figure 8.6 A two-dimensional array with four rows and five columns stored in row major order

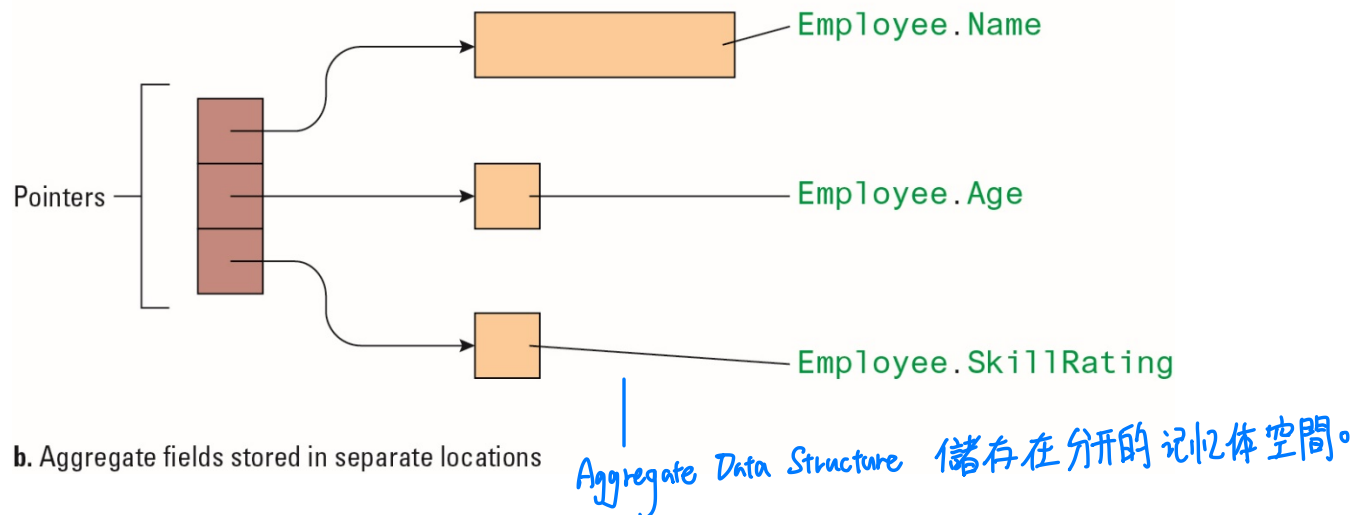
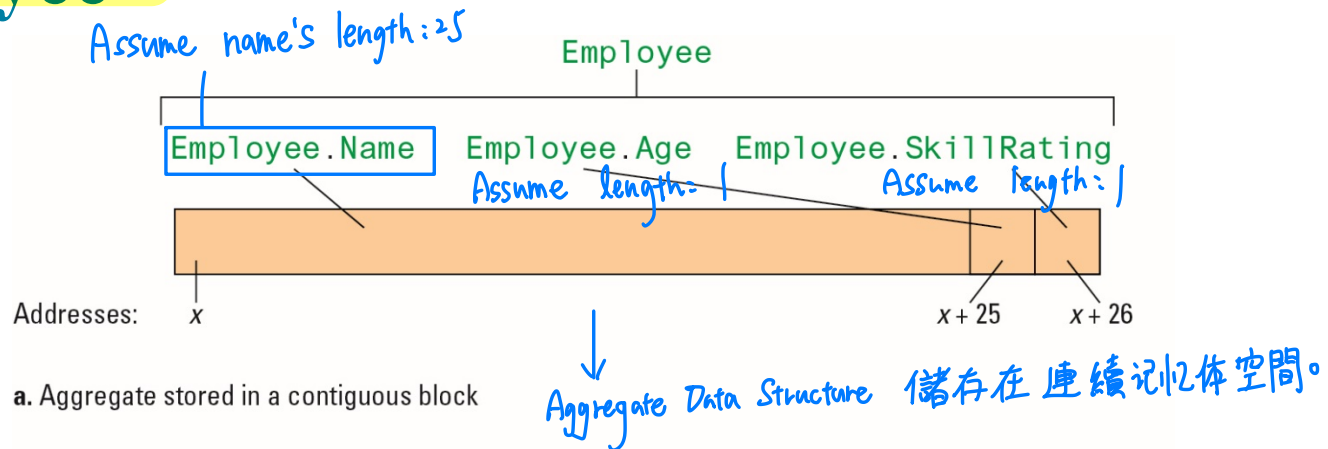


Storing Aggregates 聚合資料

field 可被一个接一个的
/ 存在連續的區塊中

- Fields can be stored one after the other in a contiguous block:
- Memory cell address of each field can be computed
- Fields can be stored in separate locations identified by pointers
 - \ 每个 field 可以存在分开的位址上，
透过指標指向下一个位址

Figure 8.7 Storing the aggregate type Employee



Storing Lists

- **Contiguous list:** List in which entries are stored in an array
連續 list: 存在連續 memory 空間
- **Linked list:** List in which entries are linked by pointers
鏈結串列: 資料存在不連續 memory 空間, 用 pointer 指向
- **Head pointer:** Pointer to first entry in list
頭指標
- **null:** A “non-pointer” value used to indicate end of list

Figure 8.8 Names stored in memory as a contiguous list

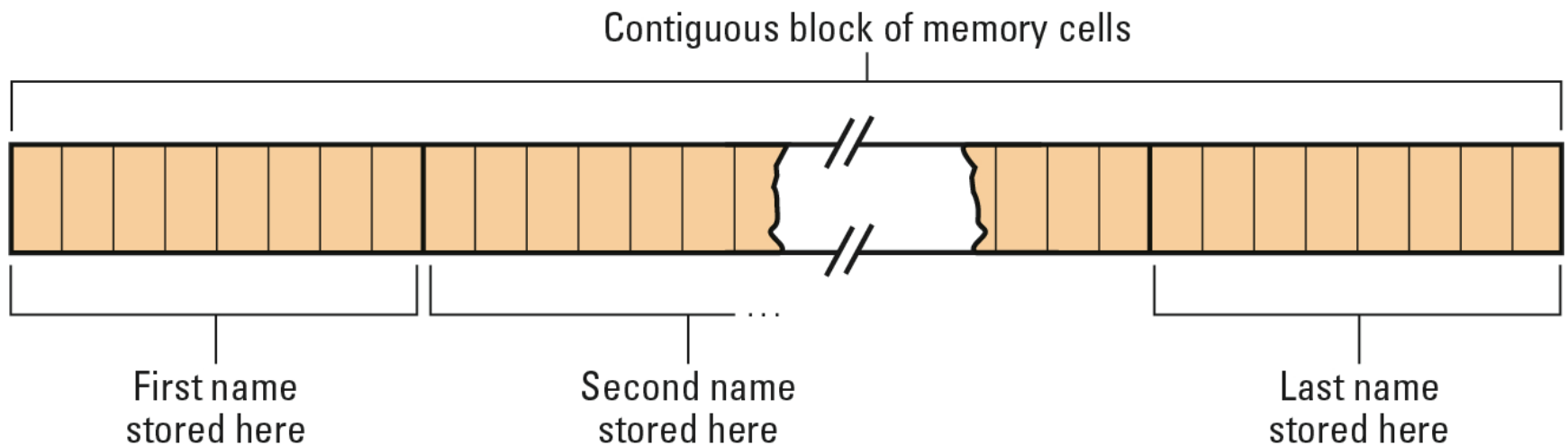


Figure 8.9 The structure of a linked list

鏈結串列實作

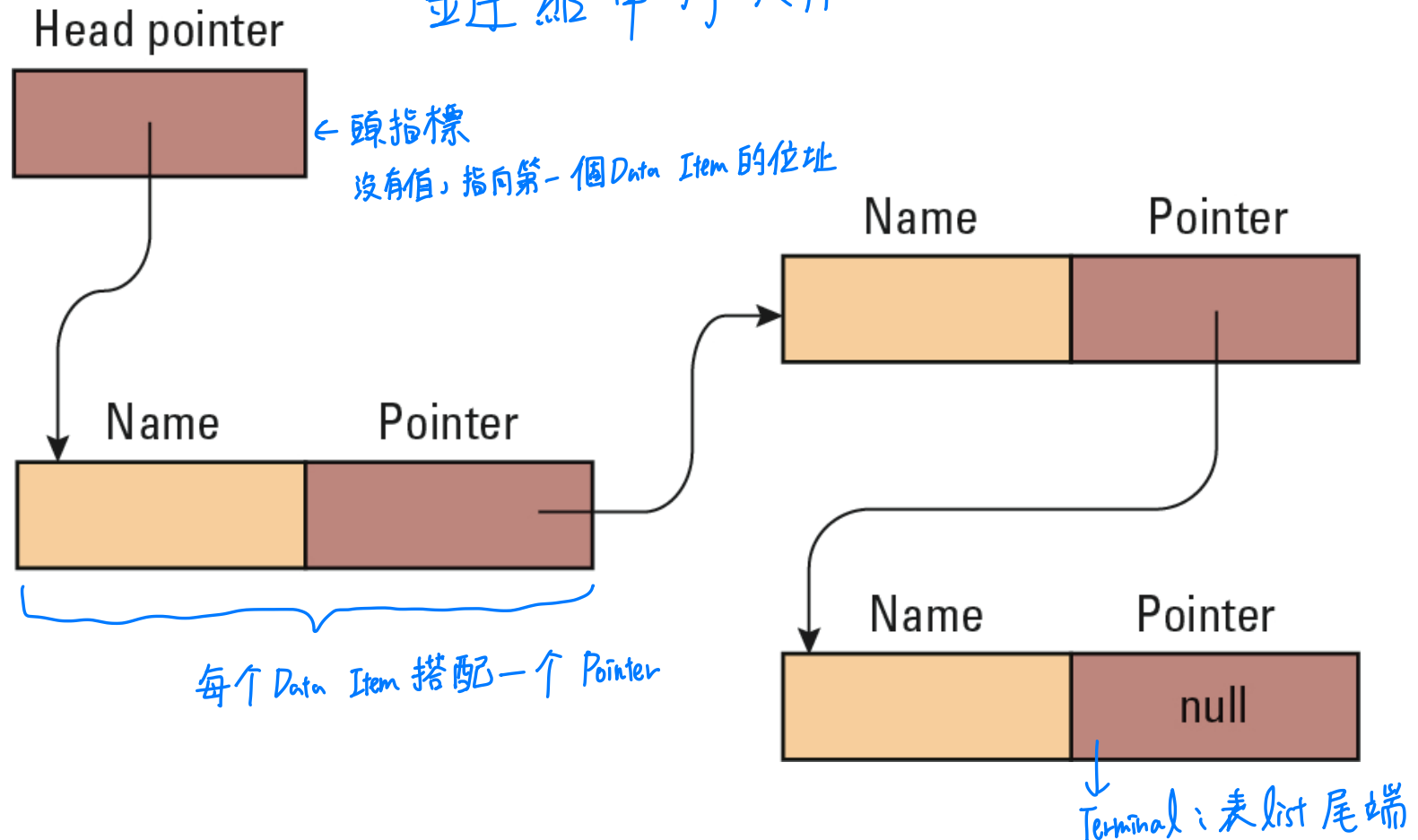


Figure 8.10 Deleting an entry from a linked list

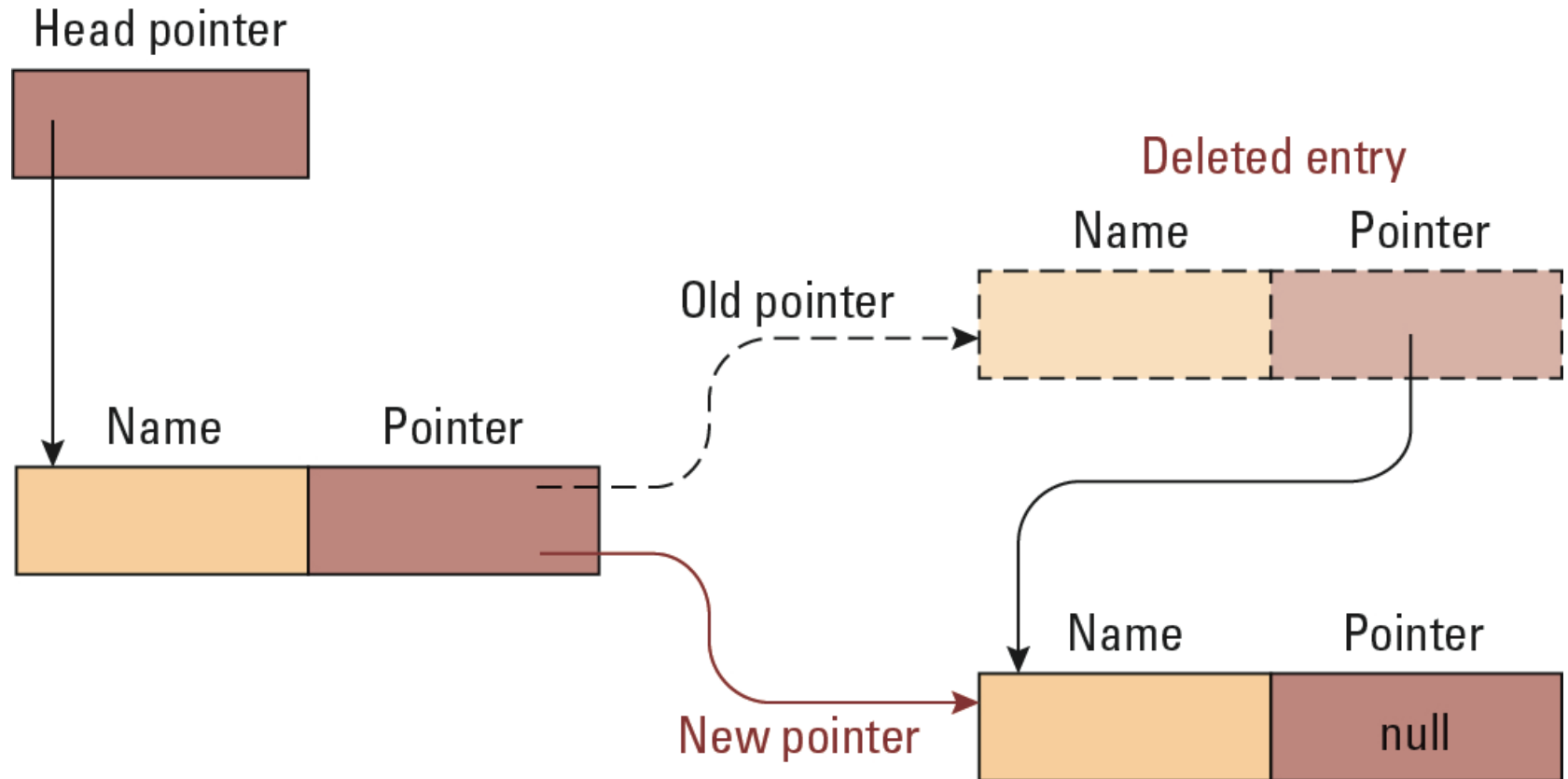
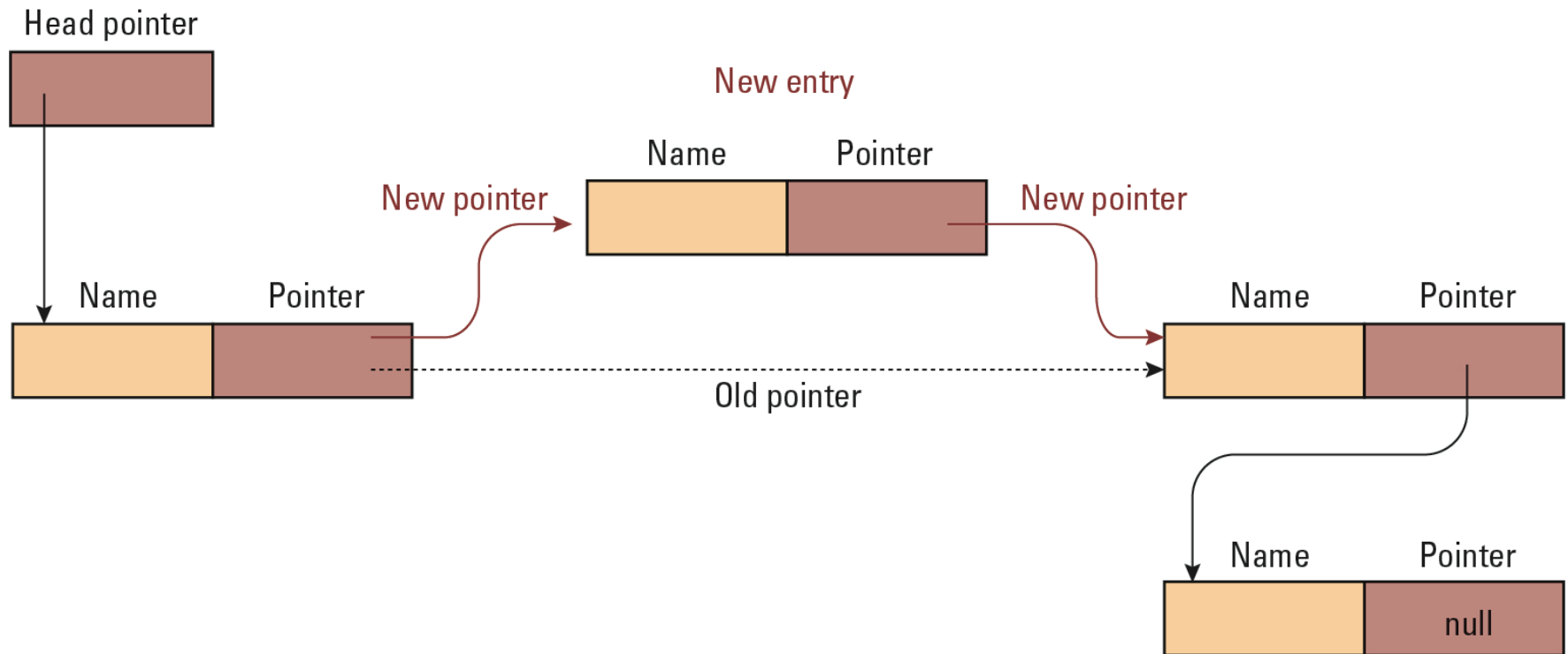


Figure 8.11 Inserting an entry into a linked list



Storing Stacks and Queues

- Stacks usually stored as contiguous lists 通常存在連續空間
- Queues usually stored as **Circular Queues** 環狀的結構!
 - Stored in a contiguous block in which the first entry is considered to follow the last entry → 第一個元素在第一個元素的後面(環狀!)
 - Prevents a queue from crawling out of its allotted storage space
防止 Access 到預分配儲存空間 外的地方。

Figure 8.12 A stack in memory

Stack: LIFO

先進後出

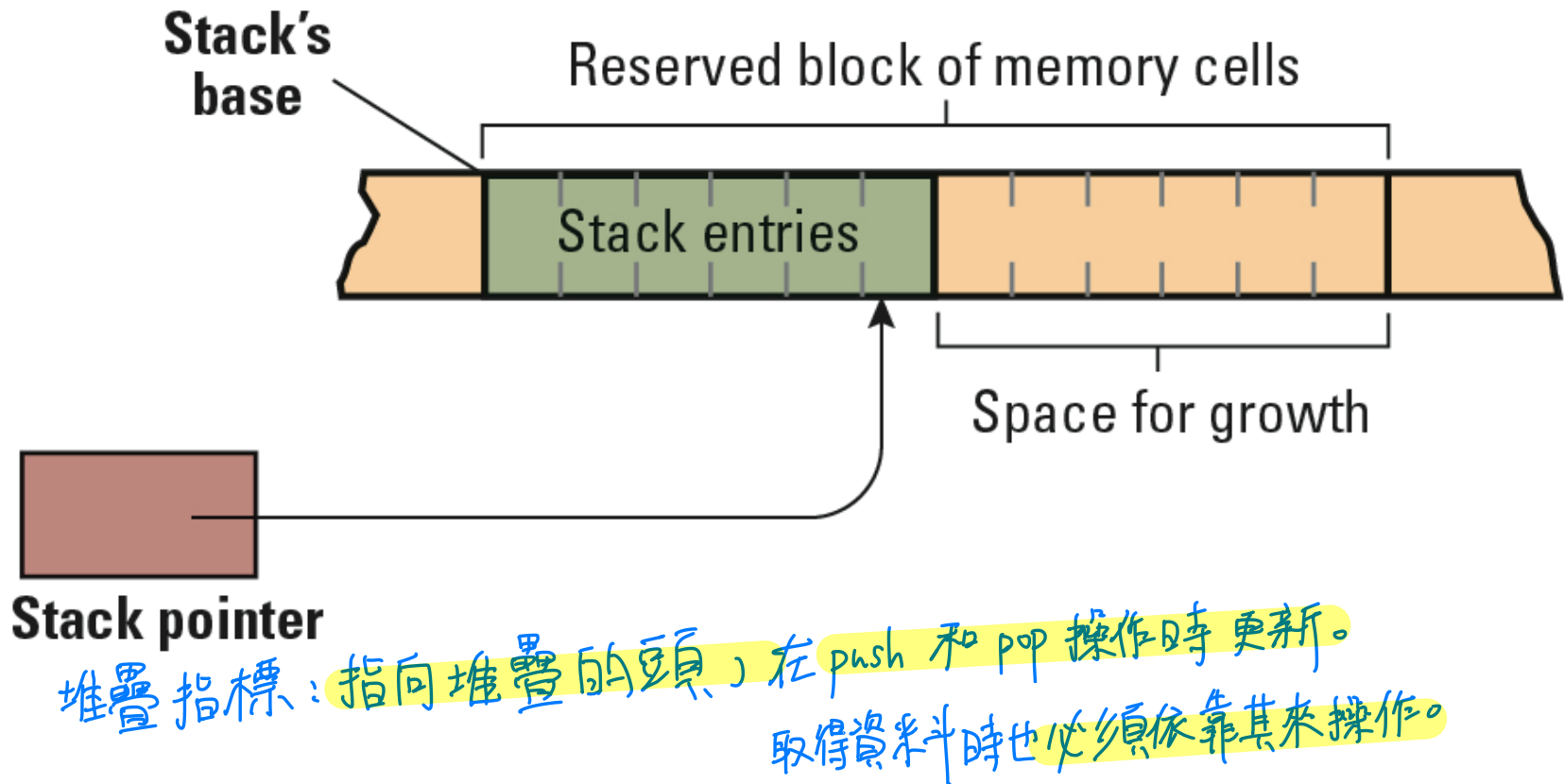
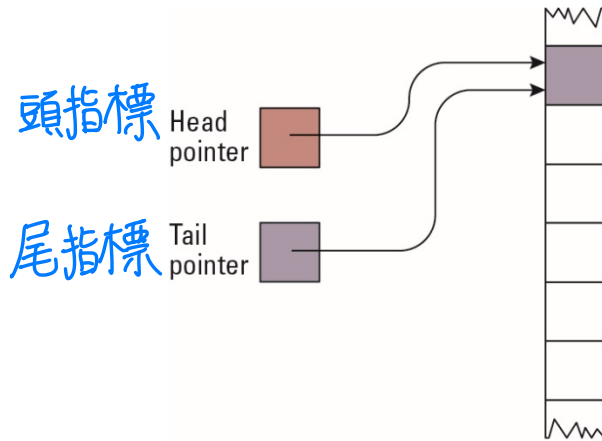


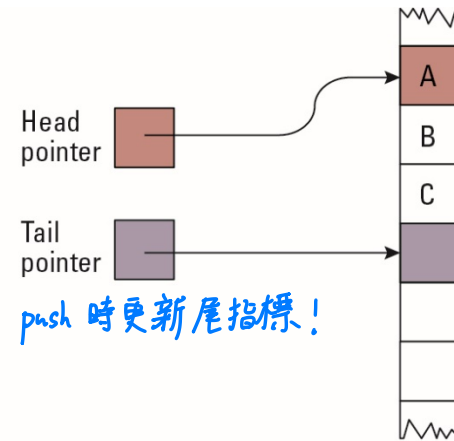
Figure 8.13 A **queue** implementation with head and tail pointers

隊列!

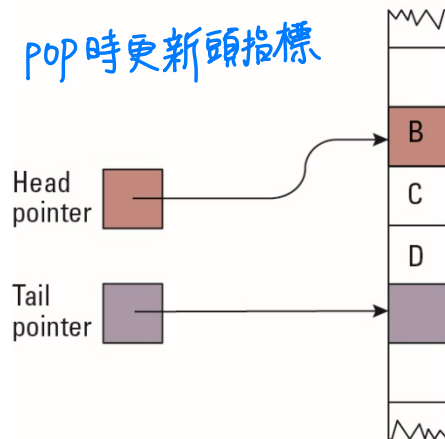
Queue: FIFO
先進先出



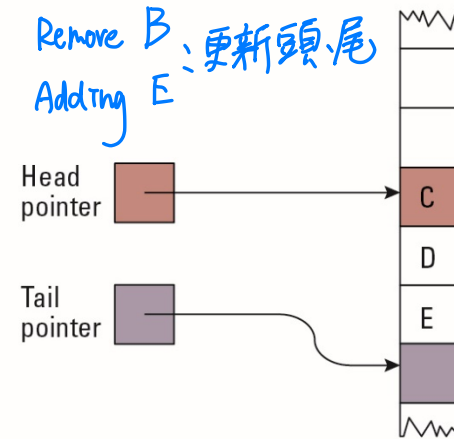
a. Empty queue



b. After inserting entries A, B, and C



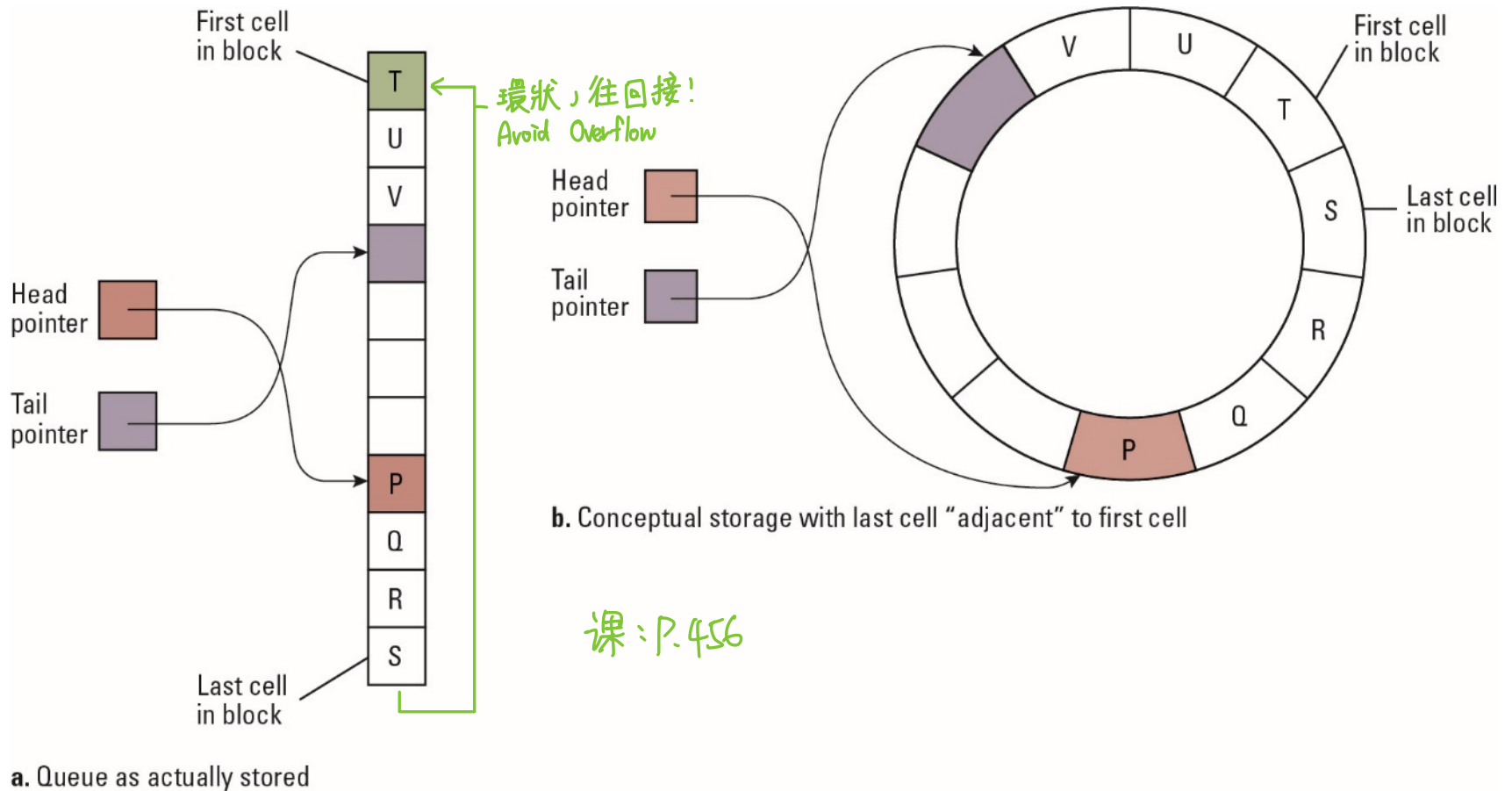
c. After removing A and inserting D



d. After removing B and inserting E

一直操作下去
可能导致 Overflow!
<solution>
環狀!
(Next Page >>)

Figure 8.14 A circular queue containing the letters P through V



Storing Binary Trees

- **Linked structure**

- Each node = data cells + **two child pointers**

- Accessed via a **pointer to root node**

Now Node Data	left Child Pointers	Right Child Pointers
---------------------	---------------------------	----------------------------

指向 left child node & Right child node.

- **Contiguous array structure**

- $A[1] = \text{root node}$

- $A[2], A[3] = \text{children of } A[1]$

- $A[4], A[5], A[6], A[7] = \text{children of } A[2] \text{ and } A[3]$



linked storage System

Array Method (會有空間上的浪費)

Due to Binary tree restricted the **Child of node** max amount is 2, we can use array to save binary tree

Figure 8.15 The structure of a node in a binary tree

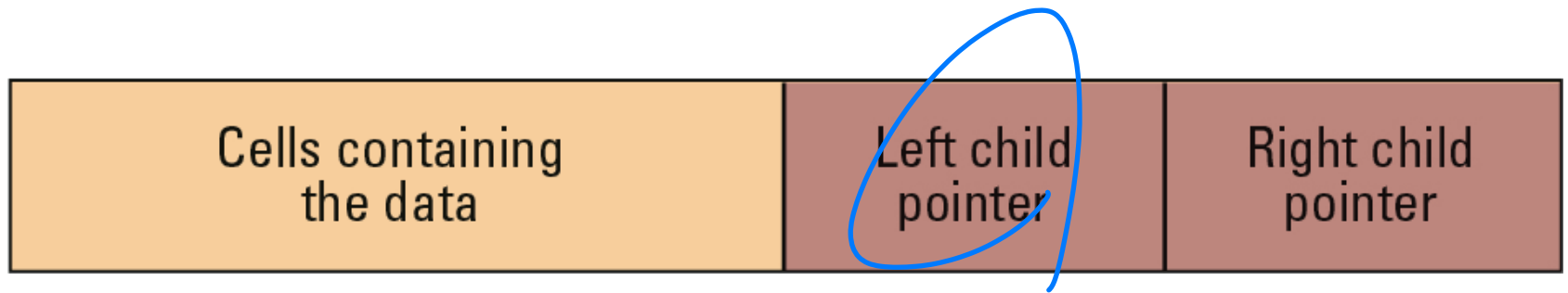
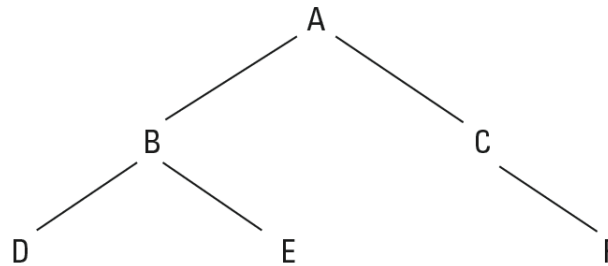


Figure 8.16 The conceptual and actual organization of a binary tree using a linked storage system

Conceptual tree



Actual storage organization

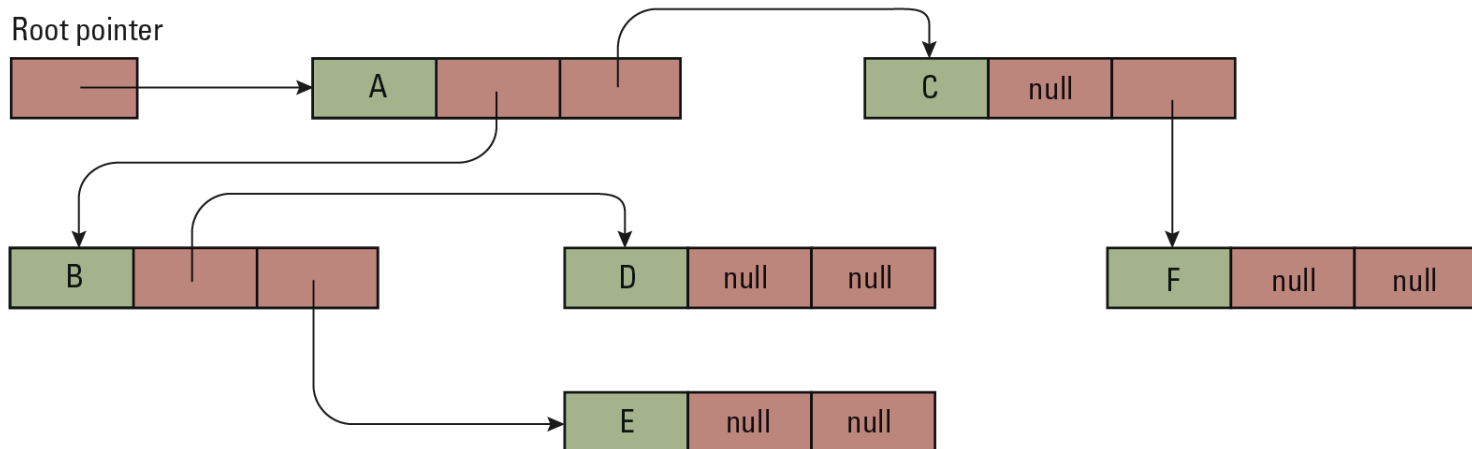
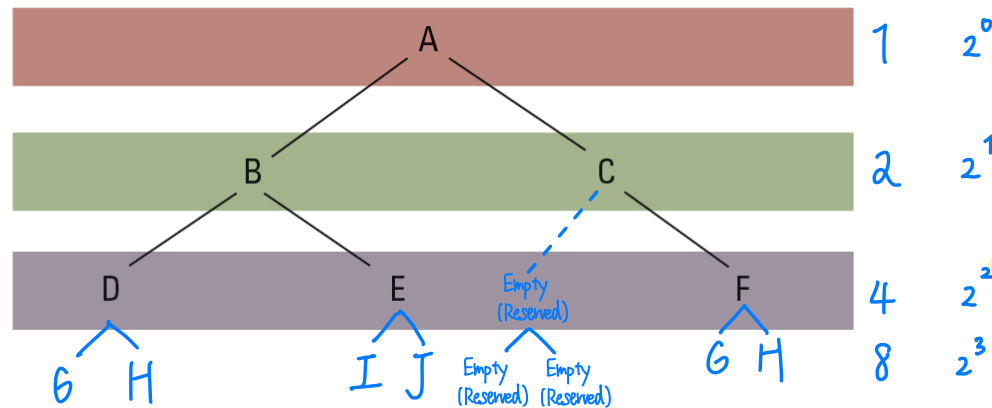


Figure 8.17 A tree stored without pointers

Conceptual tree



每一層所需的
元素最大是二的
冪次

Actual storage organization

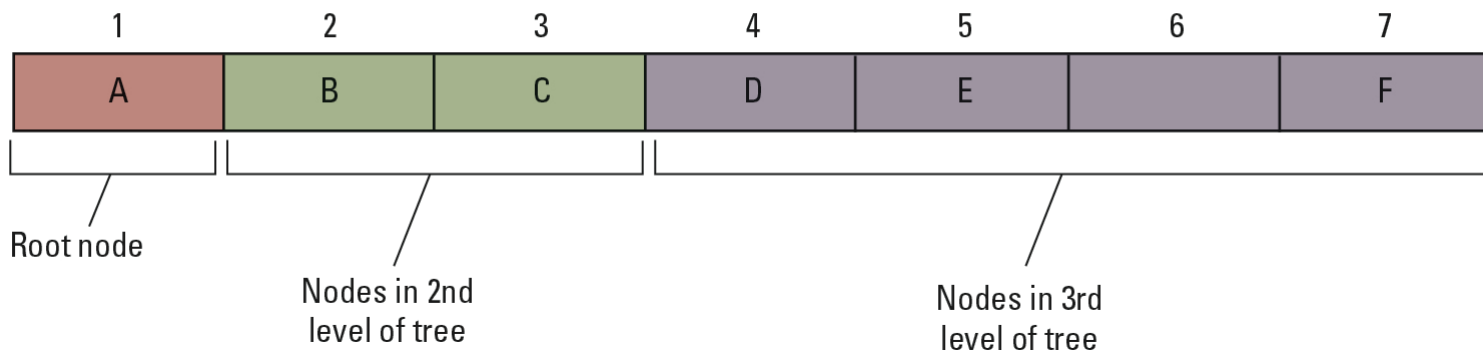
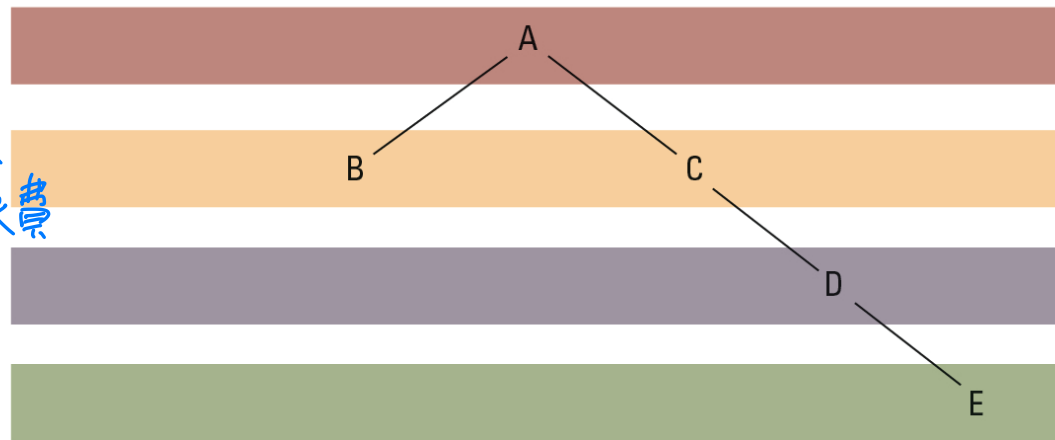


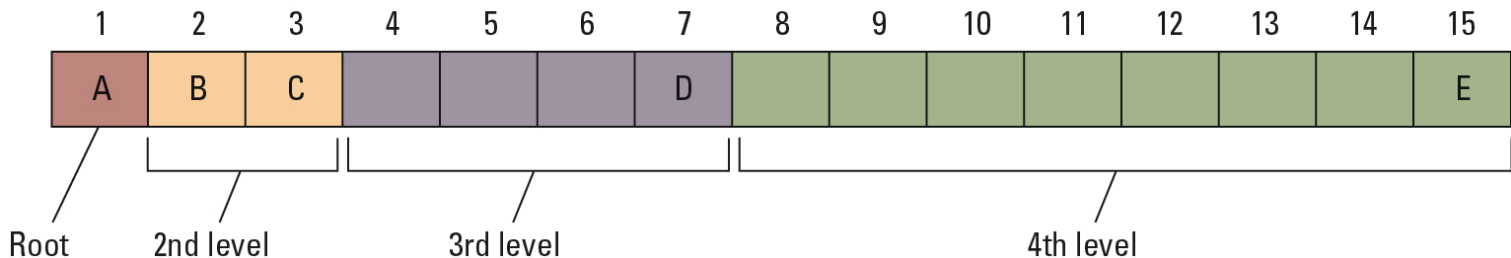
Figure 8.18 A sparse, unbalanced tree shown in its conceptual form and as it would be stored without pointers

Conceptual tree

不平衡可能导致
记忆体空间的浪费



Actual storage organization



Manipulating Data Structures

- Ideally, a data structure should be manipulated solely by pre-defined functions → 理想情況下資料結構應只使用預定義函式來操作
- Example: A list typically has a function `insert` for inserting new entries
- The data structure along with these functions constitutes a complete abstract tool

Figure 8.19 A function for printing a linked list

```
def PrintList (List):  
    CurrentPointer = List.Head  
    while (CurrentPointer is not None):  
        print(CurrentPointer.Value)  
        CurrentPointer = CurrentPointer.Next
```

8.4 A Short Case Study

Problem:

Construct an abstract tool consisting of a list of names in alphabetical order along with the operations: search, print, and insert.

课: P. 462

Figure 8.20 The letters A through M arranged in an ordered tree

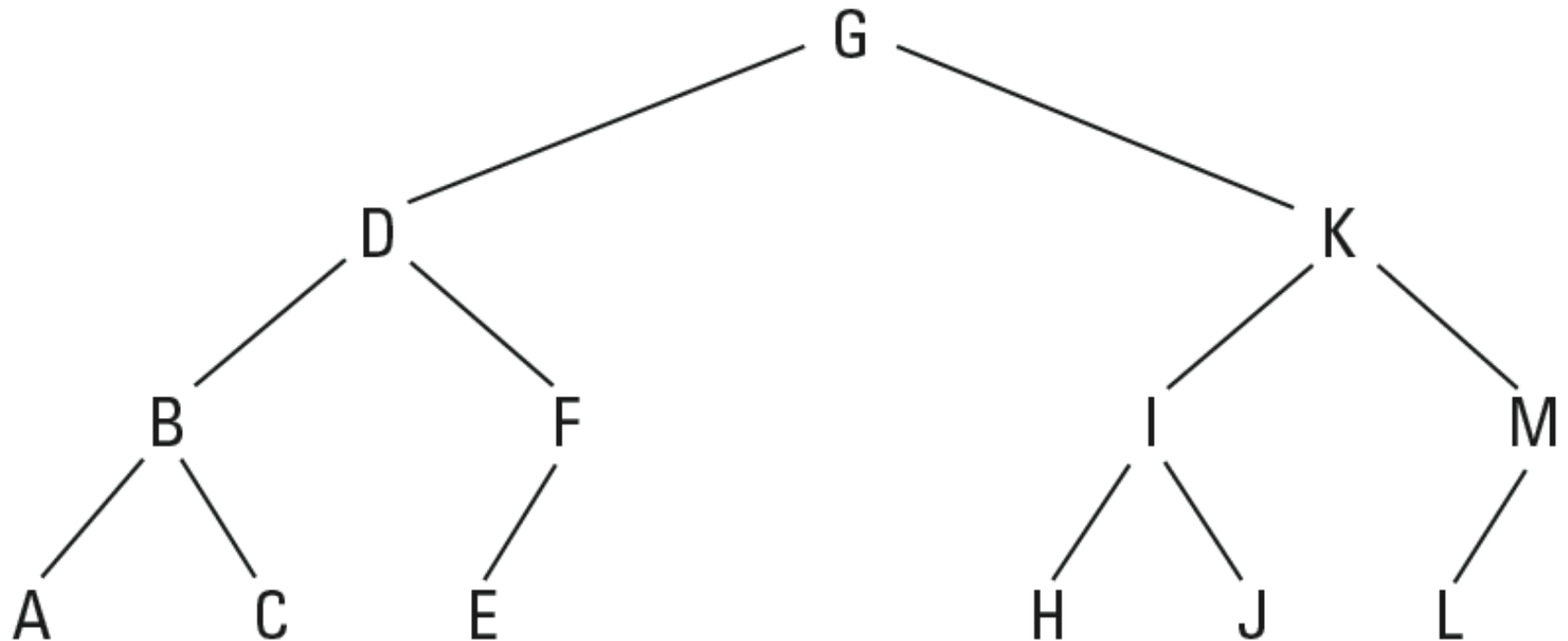


Figure 8.21 The binary search as it would appear if the list were implemented as a linked binary tree

```
def Search (Tree, TargetValue):  
    if (Tree is None):  — 結点不存在  
        return None    # Search failed  
    elif (TargetValue == Tree.Value):  — 找到  
        return Tree     # Search succeeded  
    elif (TargetValue < Tree.Value):  → 左子樹  
        # Continue search in left subtree  
        return Search(Tree.Left, TargetValue)  
    elif (TargetValue > Tree.Value):  → 右子樹  
        # Continue search in right subtree  
        return Search(Tree.Right, TargetValue)
```

Figure 8.22 The successively smaller trees considered by the function in Figure 8.21 when searching for the letter J

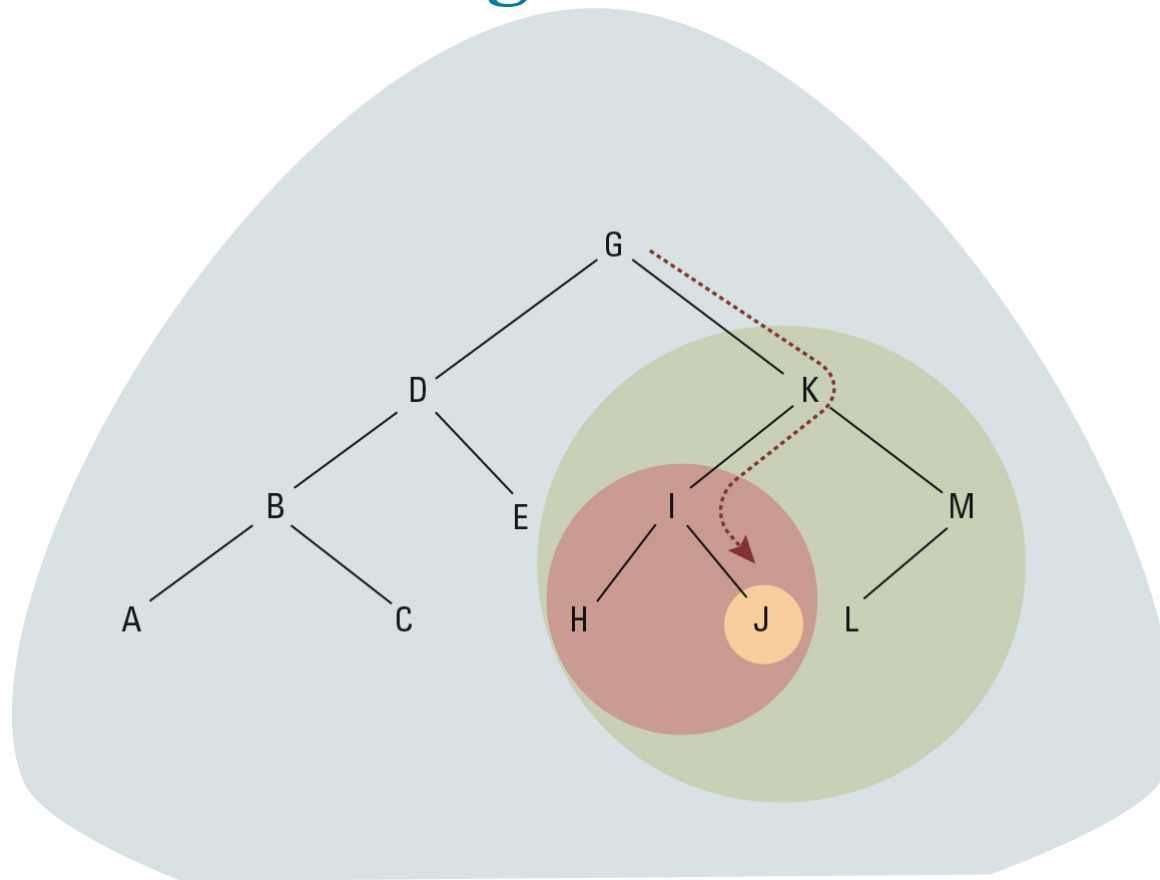


Figure 8.23 Printing a search tree in alphabetical order

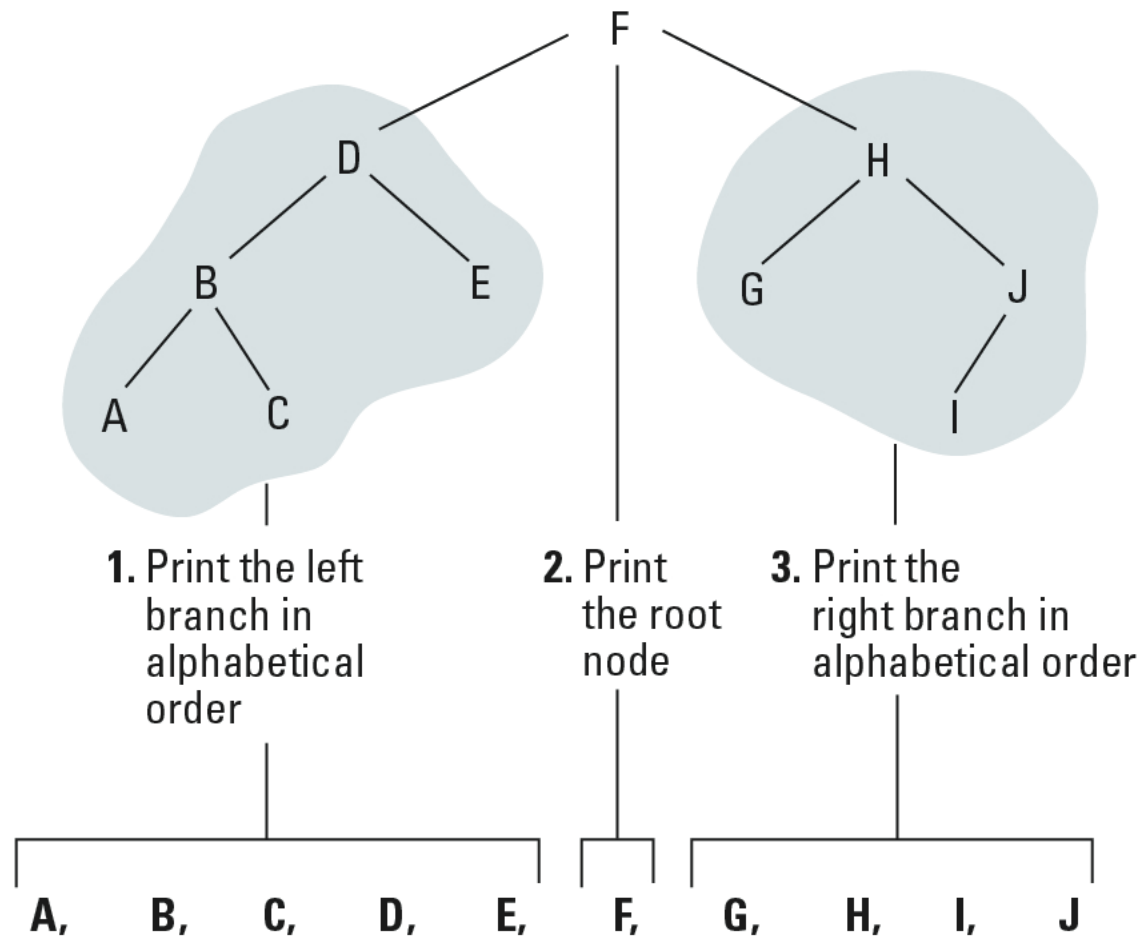
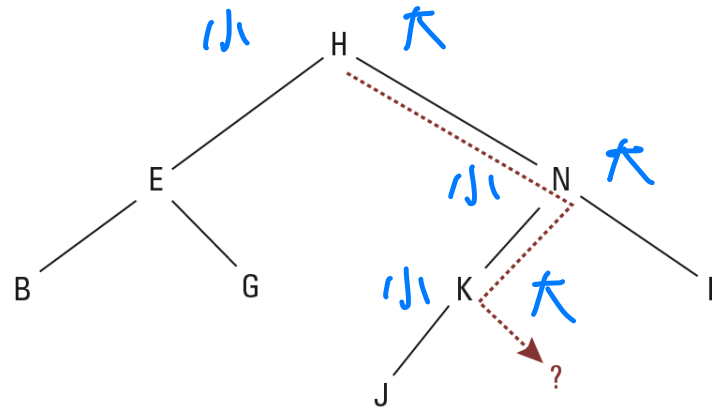


Figure 8.24 A function for printing the data in a binary tree

```
def PrintTree (Tree):  
    if (Tree is not None):  
        PrintTree(Tree.Left)  
        print(Tree.Value)  
        PrintTree(Tree.Right)
```

Figure 8.25 Inserting the entry M into the list B, E, G, H, J, K, N, P stored as a tree

a. Search for the new entry until its absence is detected



b. This is the position in which the new entry should be attached

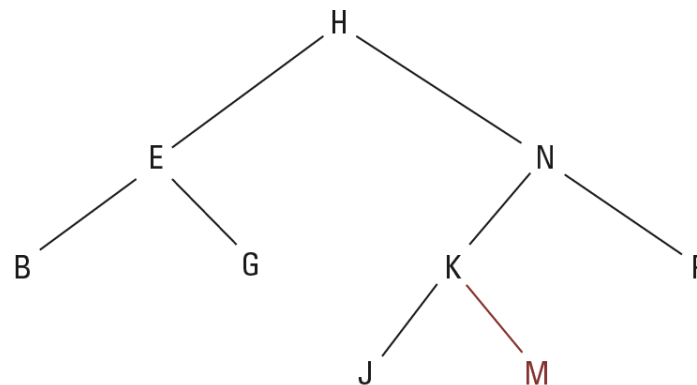


Figure 8.26 A function for inserting a new entry in a list stored as a binary tree

```
def Insert(Tree, NewValue):  
    if (Tree is None):  
        # Create a new leaf with NewValue → 建新結点, 插入值  
        Tree = TreeNode()  
        Tree.Value = NewValue  
    elif (NewValue < Tree.Value): → 小於, 往左子樹走  
        # Insert NewValue into the left subtree  
        Tree.Left = Insert(Tree.Left, NewValue)  
    elif (NewValue > Tree.Value): → 大於, 往右子樹走  
        # Insert NewValue into the right subtree  
        Tree.Right = Insert(Tree.Right, NewValue)  
    else:  
        # Make no change  
    return Tree
```

8.5 Customized Data Types 自订资料形态

Primitive Types:

adj. 原始的

integer, float, character, Boolean

A programmer can define customized data types to meet the needs of a particular application

⇒ Struct

User-defined Data Type

- Use an aggregate structure to define new type, in C:

```
struct EmployeeType
{
    char   Name[25];
    int    Age;
    real   SkillRating;
}
```

- Use the new type to define variables:

```
struct EmployeeType DistManager, SalesRep1;
```

adj. 原始的

Abstract Data Type

抽象化資料形態 (ADT)

- A user-defined data type that can include both data (representation) and functions (behavior)
- Example:

interface StackType

```
{  
    public int pop();  
    public int push(int item);  
    public boolean isEmpty();  
    public boolean isFull();  
}
```

* 課 P. 468

Conclusion:

Object 算一種 ADT 嗎? Yes!

8.6 Classes and Objects

類別：一種有額外附加功能的ADT

- Class: an abstract data type with extra features
 - Properties can be inherited 屬性可被繼承！
 - Constructor methods to initialize new objects / Constructor 來建構及初始化物件
 - Contents can be encapsulated 內容可被封裝！ (e.g. public, private)
- Object: an instance of a class 物件：一個類別實體化

Figure 8.27 A stack of integers implemented in Java and C#

class StackOfIntegers implements StackType

```
{  
    private int[] StackEntries = new int[20];  
    private int StackPointer = 0;  
  
    public void push(int NewEntry)  
    {  
        if (StackPointer < 20)  
            StackEntries[StackPointer++] = NewEntry;  
    }  
  
    public int pop()  
    {  
        if (StackPointer > 0) return StackEntries[--StackPointer];  
        else return 0;  
    }  
  
    public boolean isEmpty()  
    {  
        return (StackPointer == 0);  
    }  
  
    public boolean isFull()  
    {  
        return (StackPointer >= MAX);  
    }  
}
```

8.7 Pointers in Machine Language

立即定址法

- **Immediate addressing:** Instruction contains the data to be accessed
指令中已經包含要存取的材料

直接定址法

- **Direct addressing:** Instruction contains the address of the data to be accessed
指令中包含要存取資料的記憶體位置

間接定址法

- **Indirect addressing:** Instruction contains the location of the address of the data to be accessed
指令中包含指向欲存取資料記憶體位置的位置
(指向位址的位址)

課 P. 474

Figure 8.28 Our first attempt at expanding the machine language in Appendix C to take advantage of pointers

間接定址法
Indirect Addressing.

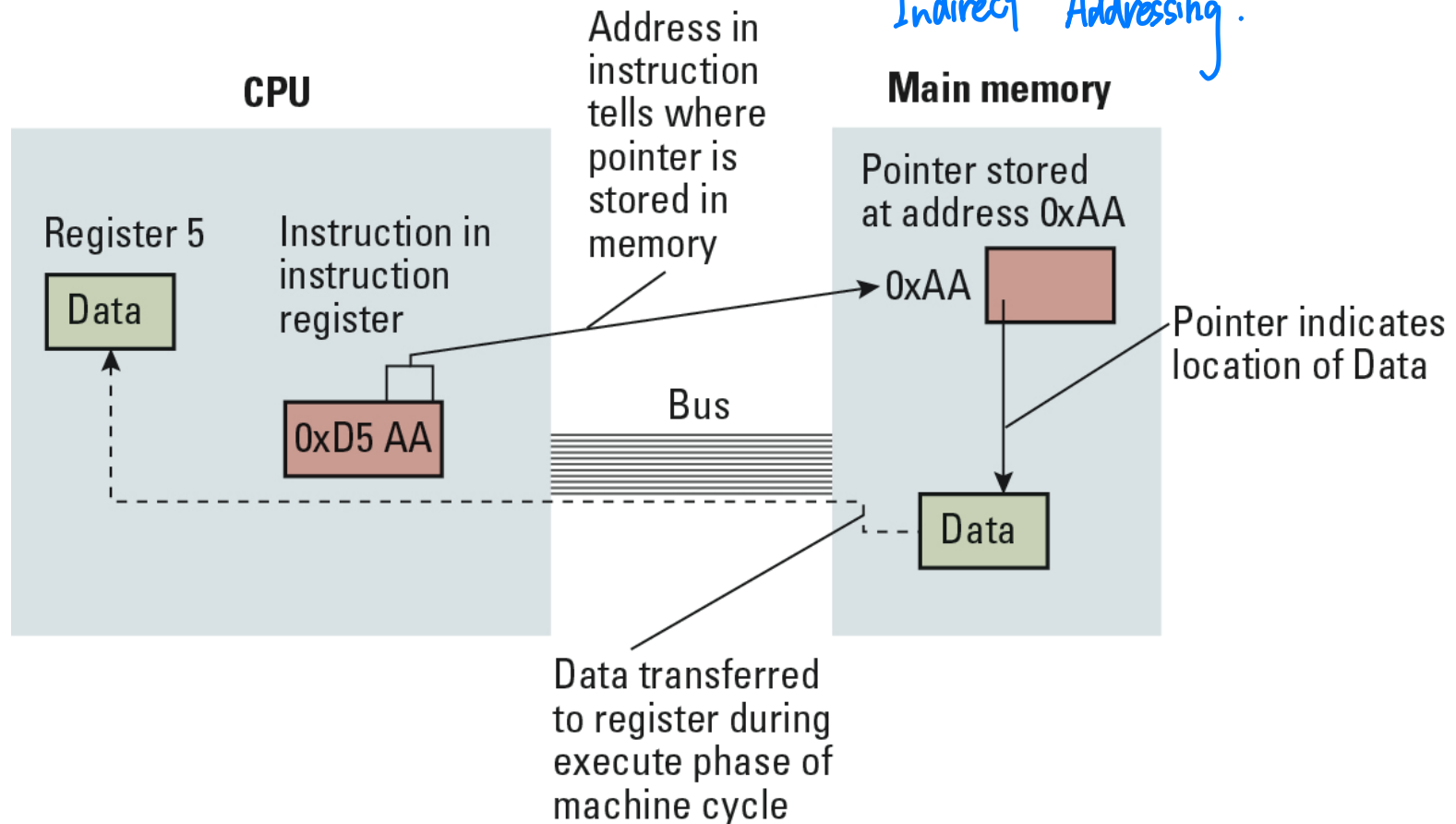


Figure 8.29 Loading a register from a memory cell that is located by means of a pointer stored in a register

